

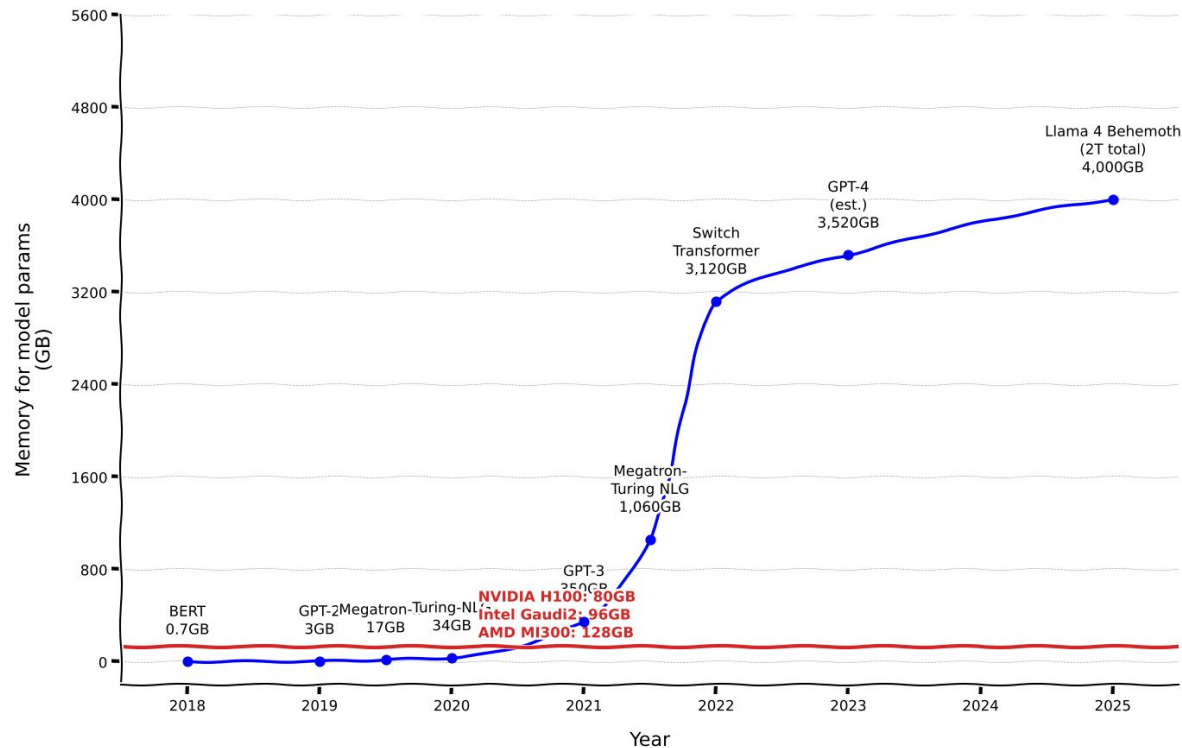


CSE 585

Divya Shekar, Sharvani Sinha, Shresth Tiwary

Scale of Modern AI

- Modern DNNs have grown to hundreds of billions of parameters.
- Training requires massive GPU clusters and can take weeks or months.
- **Problem:** At this scale, hardware and software failures are not an exception—they are the norm



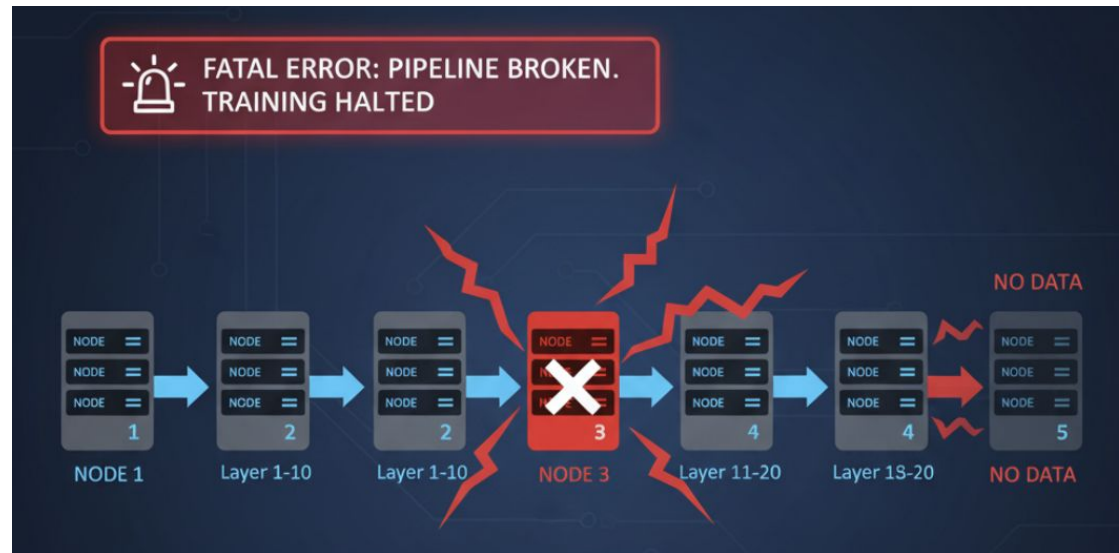
The Reality of Large-Scale DNN Training

Large DNNs need **multi-dimensional parallelism** (DP, TP, PP)

- Jobs run for **days/weeks** → **GPUs may fail or be reallocated**
- Current DL frameworks tightly couple jobs to GPUs → **no flexibility, not device independent**

The Impact of a Single Failure

- Synchronous training means all GPUs must work in lock-step.
- When one node fails, the **entire cluster of thousands of GPUs must idle** until it's recovered.
- This leads to massive underutilization and wasted resources, as reported by teams at Meta and HuggingFace.





Oobleck: Resilient Distributed Training of Large Models Using Pipeline Templates



Reviewing existing SOTAs

Feature	Varuna	Bamboo
Primary Parallelism Strategy	Pipeline Parallelism combined with Data Parallelism	Pipeline Parallelism
Fault Tolerance Mechanism	Continuous Checkpointing and Job Morphing	Redundant Computation
Recovery from Failure	Restarts from the last checkpoint with a reconfigured ("morphed") job	Predecessor node takes over the work of the failed node
Key Innovation	"Job Morphing" for dynamic resource adaptation	Using pipeline bubbles for low-cost redundant computation

Introducing Oobleck: A New Approach

A resilient training framework with **guaranteed fault tolerance**.

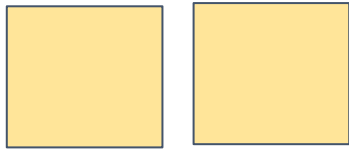
Core Idea: A planning-execution co-design.

- **Planning:** Intelligently generate **Pipeline Templates** once at the start.
- **Execution:** Use **Pipeline Replicas** for in-memory, fast recovery.

Core Idea 1: Pipeline Templates (The Blueprints)

- A **template** is a pre-computed, optimized blueprint for how to run the model on a specific number of nodes.
- Oobleck generates a library of templates for different cluster sizes (e.g., for 2-nodes, 3-nodes, 4-nodes).
- This **guarantees that any number of remaining nodes can be fully utilized** after a failure, preventing idle GPUs.

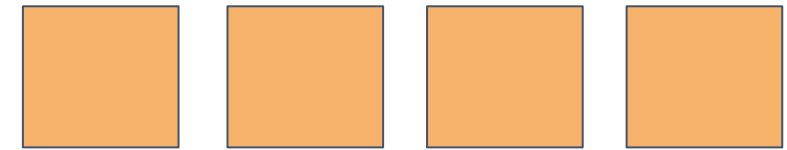
Core Idea 1: Pipeline Templates (The Blueprints)



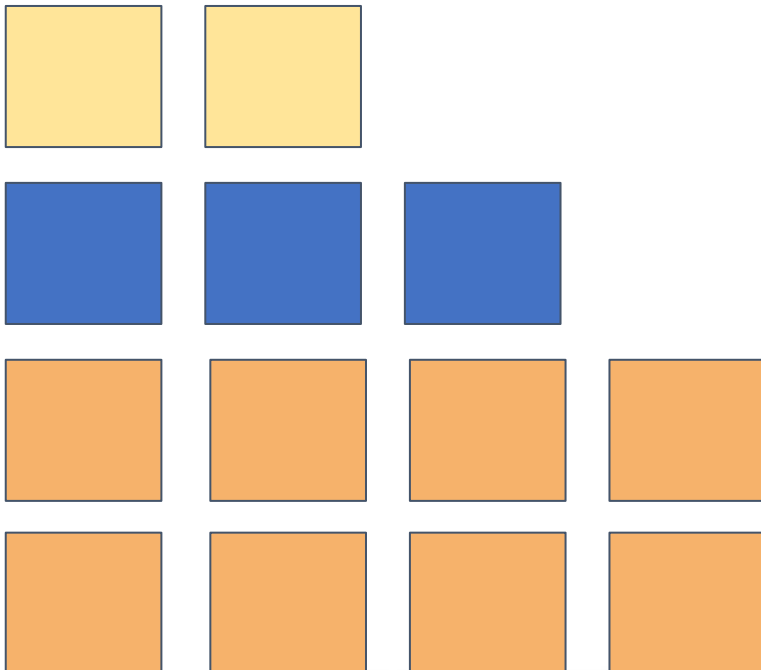
Pipeline template for 2 nodes



Pipeline template for 3 nodes



Pipeline template for 4 nodes

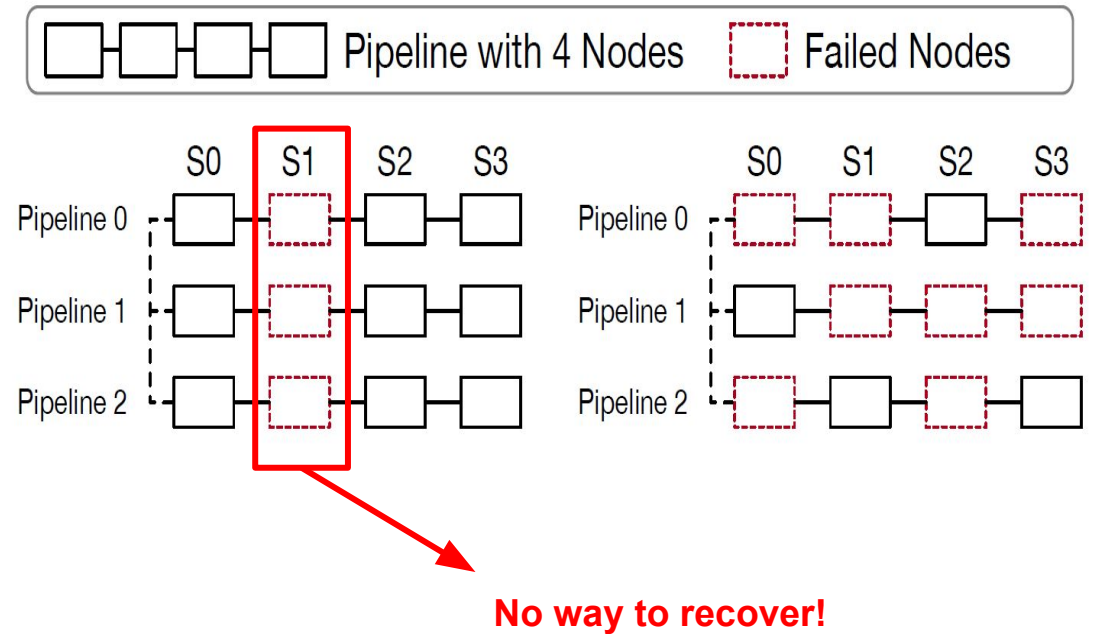


Example of parallel execution with 13 nodes!

1x pipeline from the pipeline template for 2 nodes
1x pipeline from the pipeline template for 3 nodes
2x pipeline from the pipeline template for 4 nodes

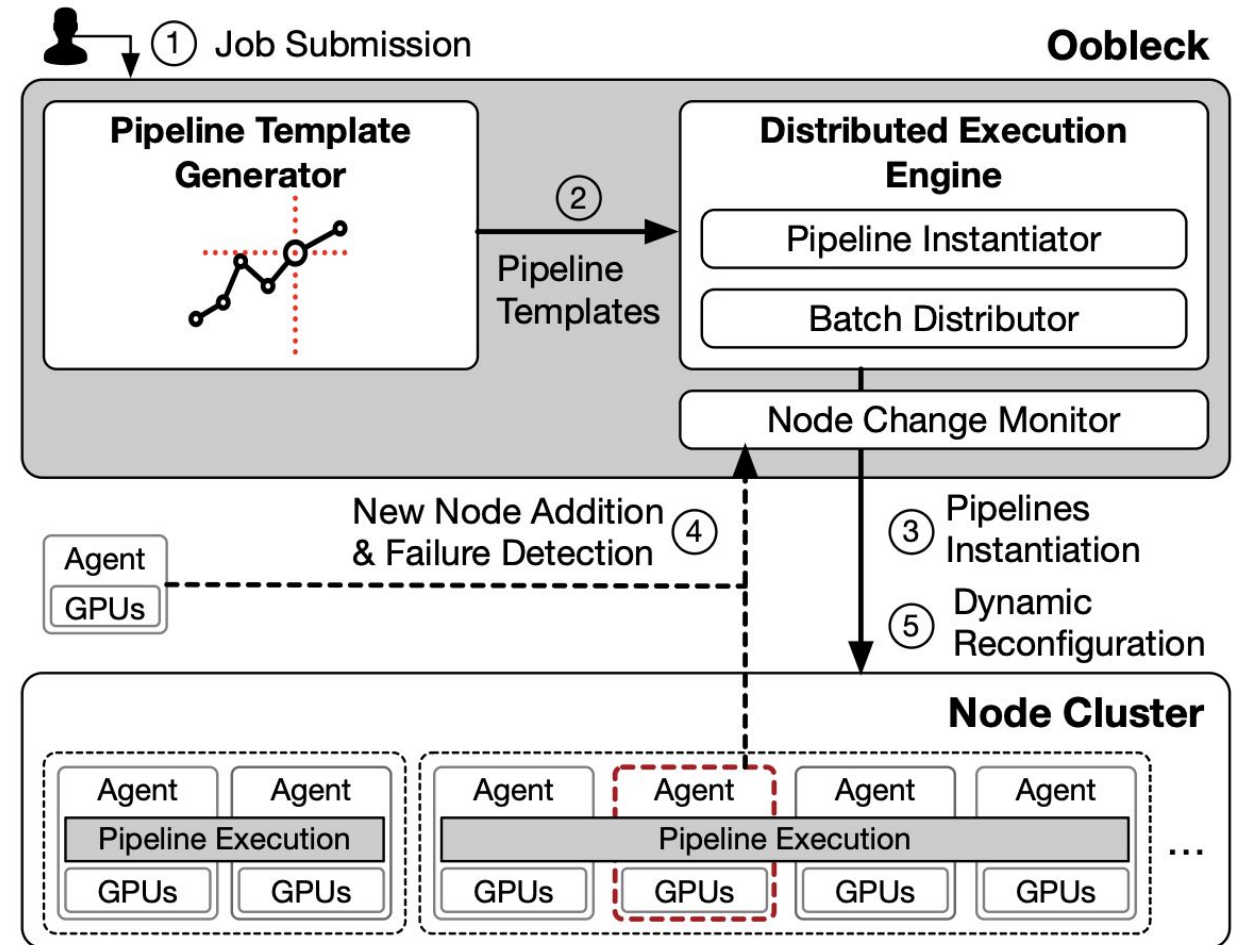
Core Idea 2: Pipeline Replicas (The Live Backups)

- Oobleck runs multiple, independent pipelines in parallel to tolerate failures.
- To tolerate **f** failures, it runs at least **f+1 replicas**.
- **The Key:** The complete, up-to-date model state always exists in the memory of a surviving replica. This is the secret to fast recovery.



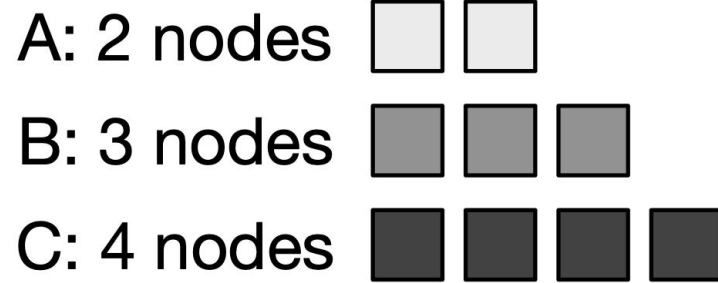
System Architecture

- 1. Pipeline Template Generator**
Generate heterogeneous pipeline templates
- 2. Distributed Execution Engine**
Instantiates pipelines from pipeline templates
Create at least $f+1$ pipeline replicas
- 3. Batch Distributor**
Calculates the number of microbatches for each pipeline
Balances execution latency



Generating Pipeline Templates

Number of Nodes (N) = 13
Model Memory Requirements
Fault Tolerance Threshold (f)



3 Pipeline Templates

Node Specification

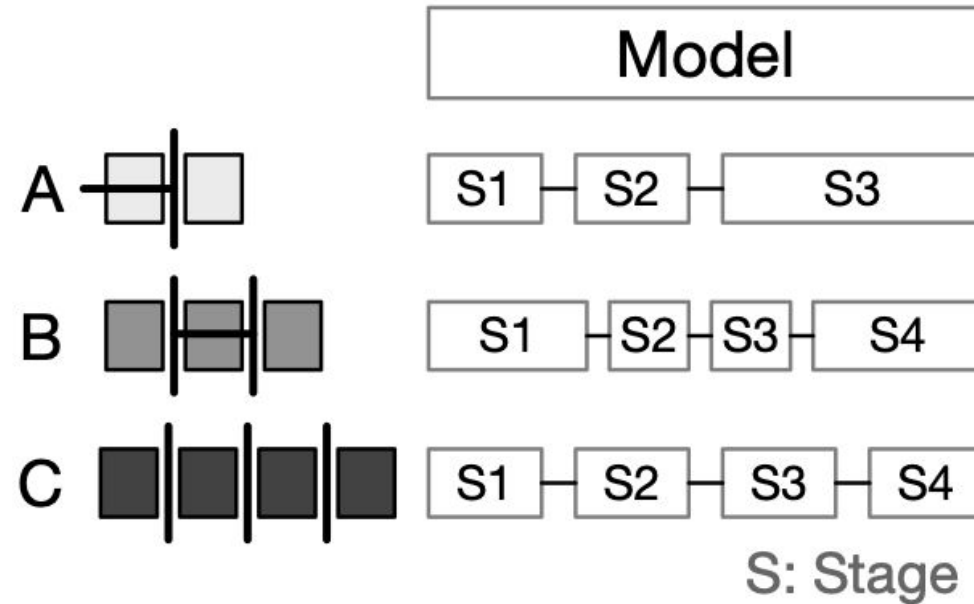
Oobleck :

- n_0 : smallest # of nodes for a pipeline
- p : # of pipeline templates
- N : total nodes
- N' : remaining nodes after failure
- f : fault tolerance

1. $p > n_0 - 1$
2. $p < N - fn_0$

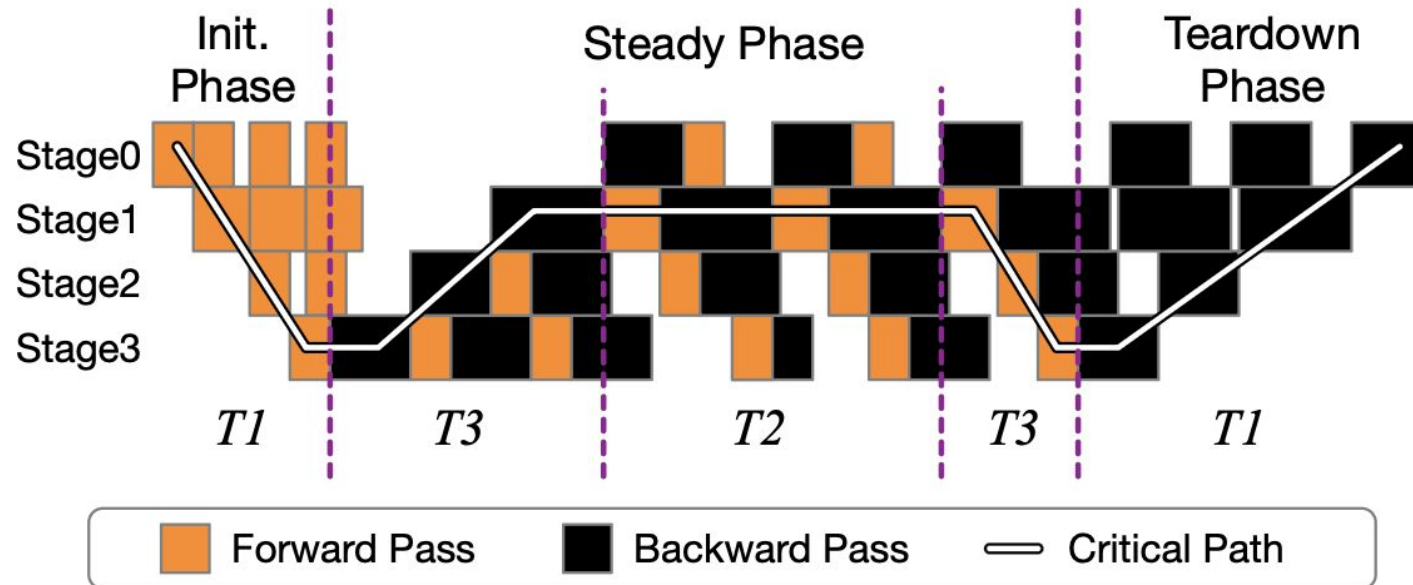
Small n_0 -> less compute time.
 N' can be linearly expressed in terms of pipeline templates

GPU-State Mapping



GPU—Stage Mapping

Algorithm

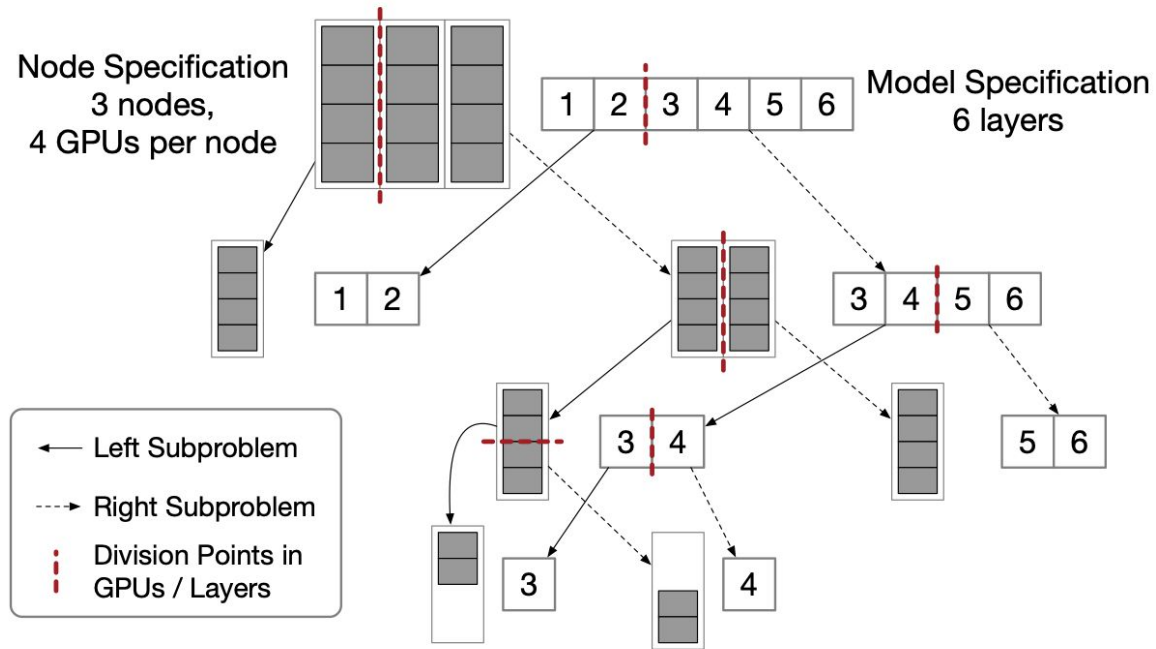


1F1B pipeline execution

$$T = T1 + T2 + T3$$

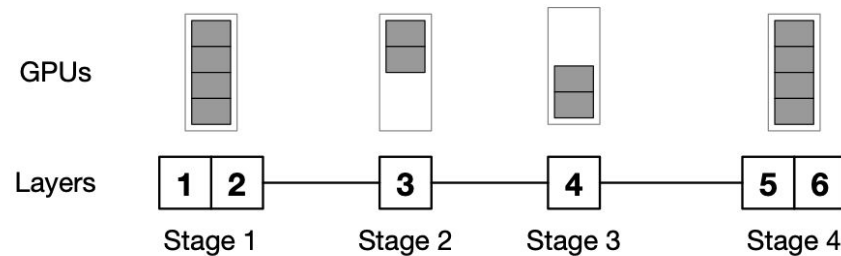
- Initialisation: 1 forward pass
- Teardown: 1 backward pass
- Steady Phase: Alternating forward and backward passes

Algorithm - Divide and Conquer



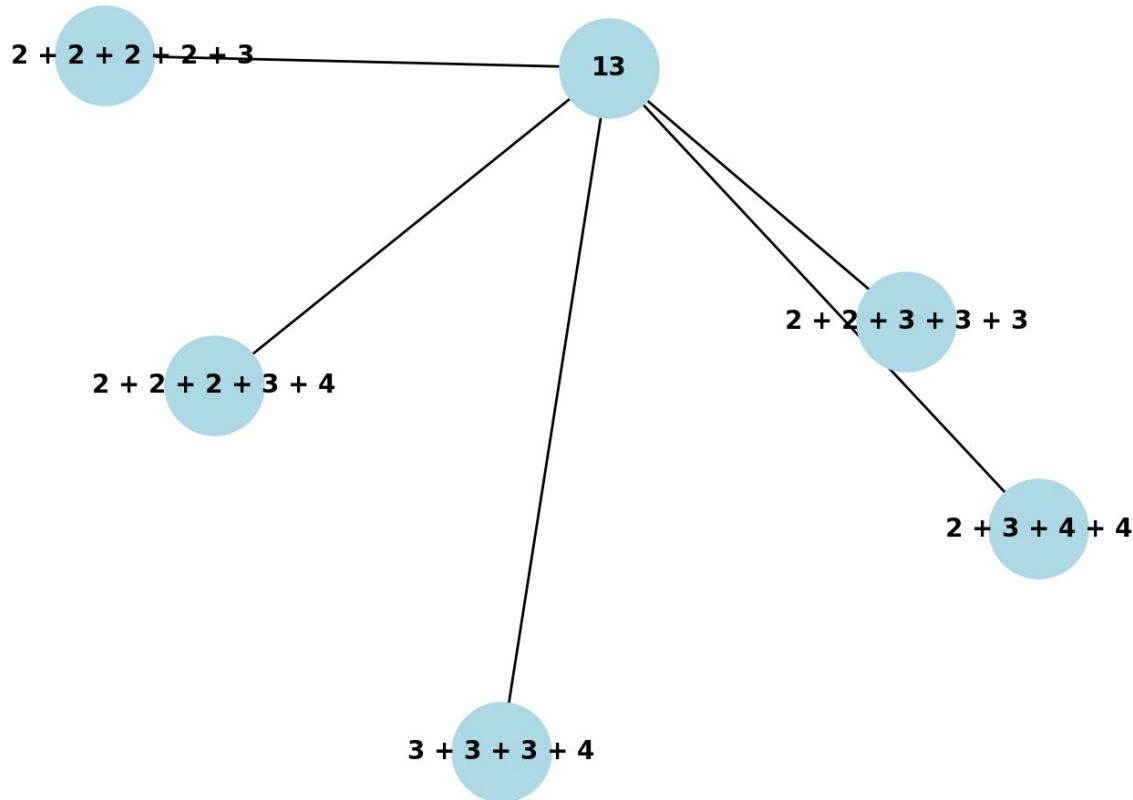
Find a mapping such that iteration time is minimized!

A Mapping Result of a 4-Stage Pipeline



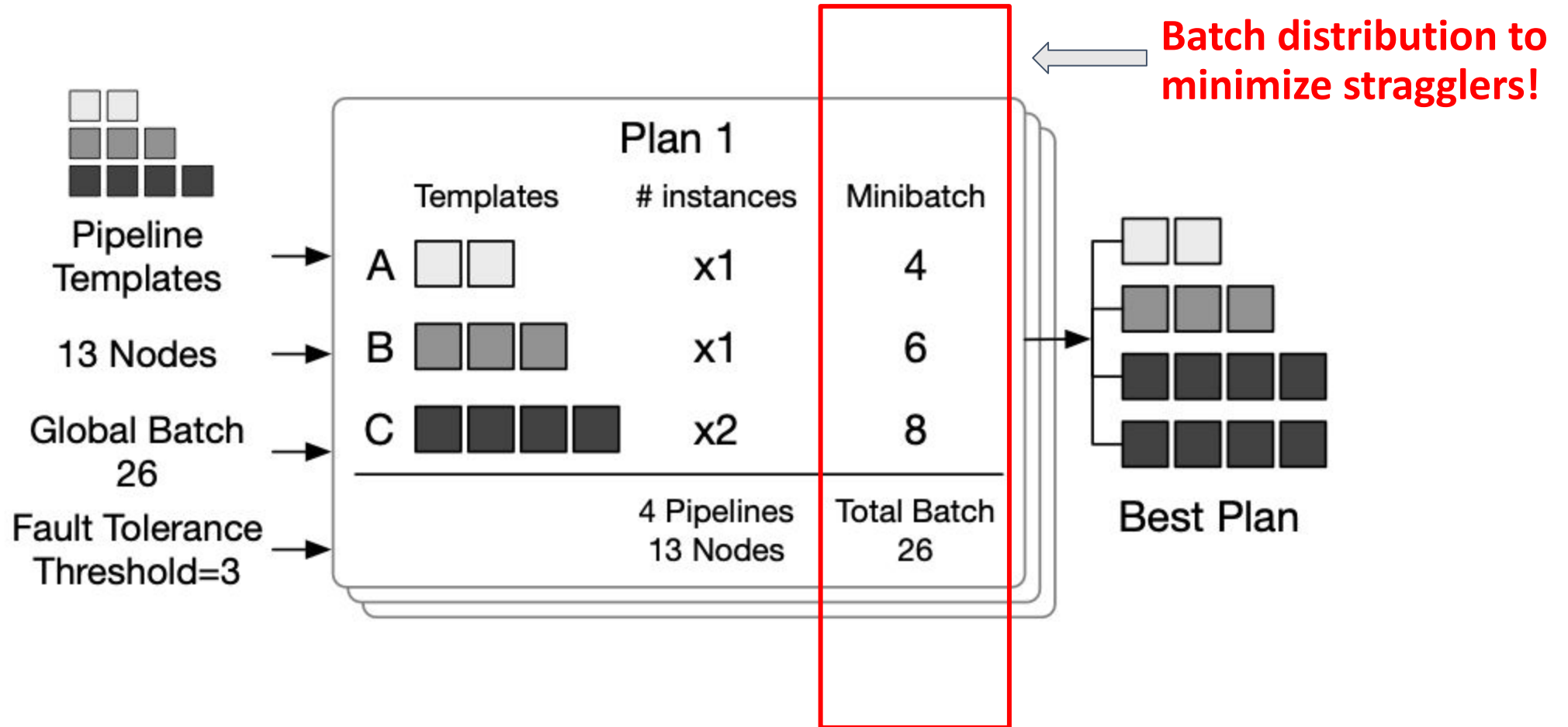
Pipeline Instantiation

Combinations of 2, 3, 4 that Sum to 13 (Order-Invariant)



**Choose a combination
that maximizes
throughput !**

Pipeline Instantiation

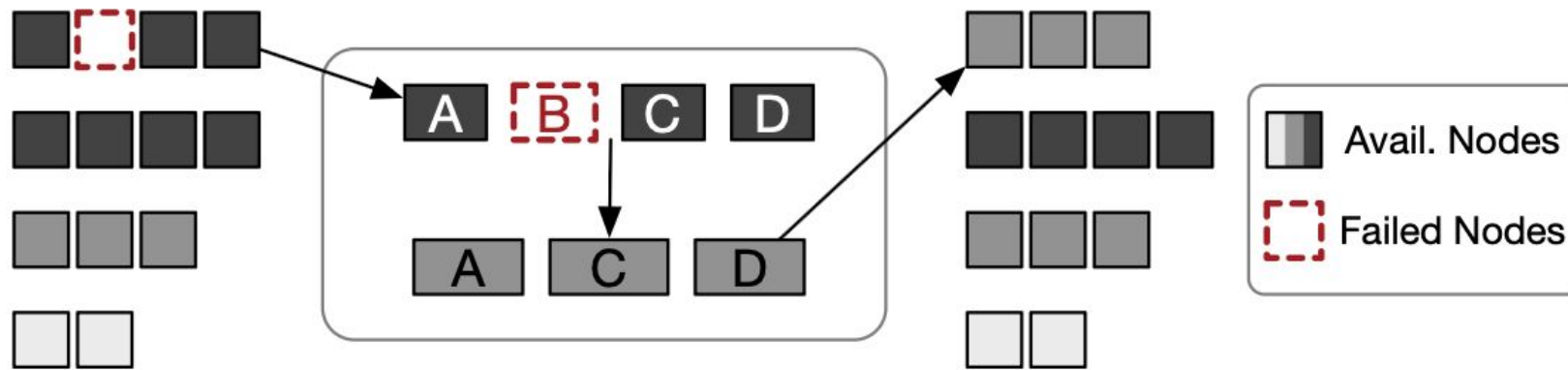


Dynamic Reconfiguration

- Upon a node failure, pipeline becomes incomplete and has missing model states
- Training is halted
- Replace with new pipelines created via **pipeline reinstantiation** using precomputed pipeline templates
- Copy missing layers from unaffected pipeline replicas
- **Batch redistribution** to maintain balanced execution

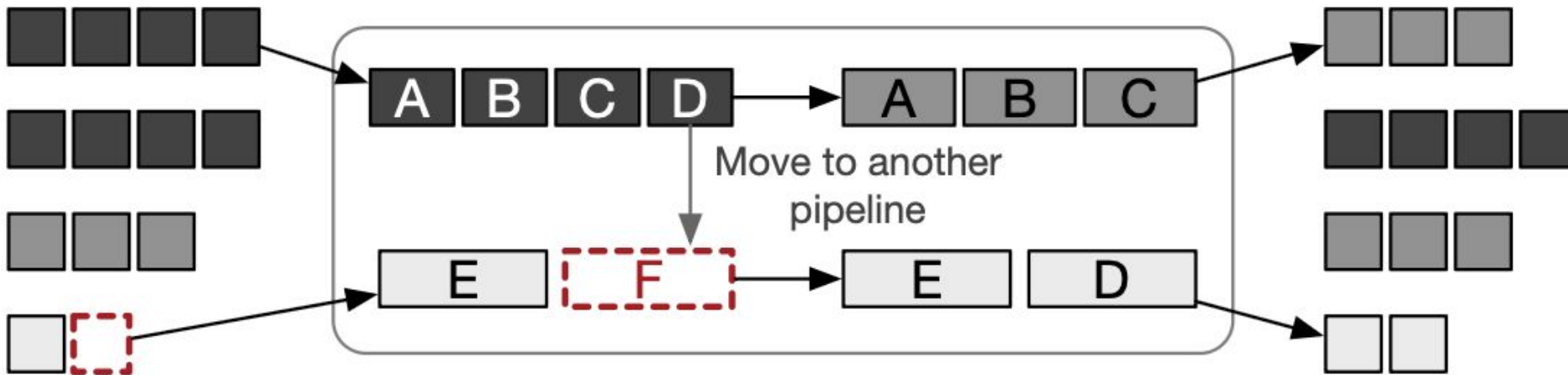
Pipeline Reinstantiation

Case 1: Simple Reinstantiation



If there is an insatiable pipeline template with remaining nodes, simple reinitiate and replace the pipeline

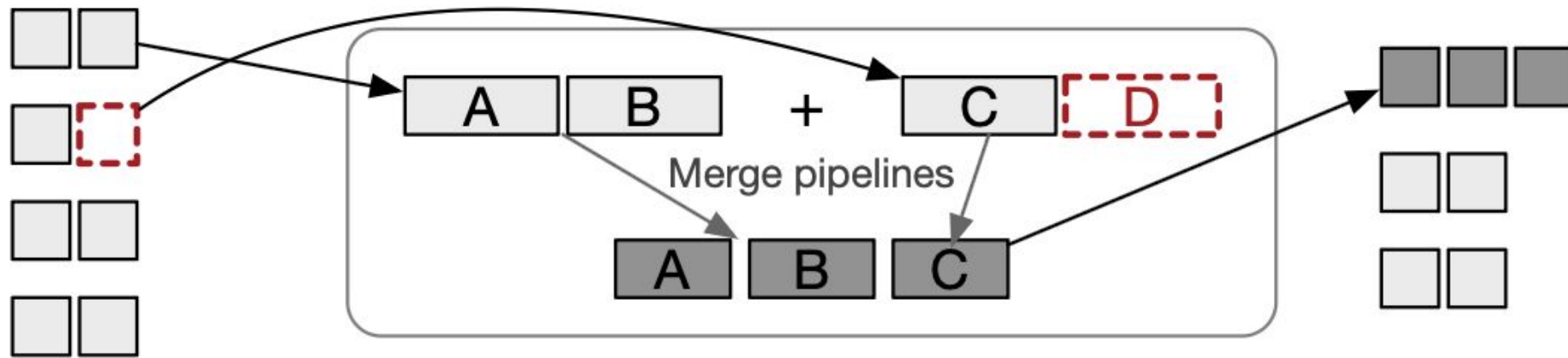
Case 2: Borrowing Nodes



If there is no instantiable pipeline template with remaining nodes, borrow nodes from other pipelines to instantiate the smallest pipeline template.

Also reinstantiate the yielding pipeline with fewer nodes.

Case 3: Merging Pipelines



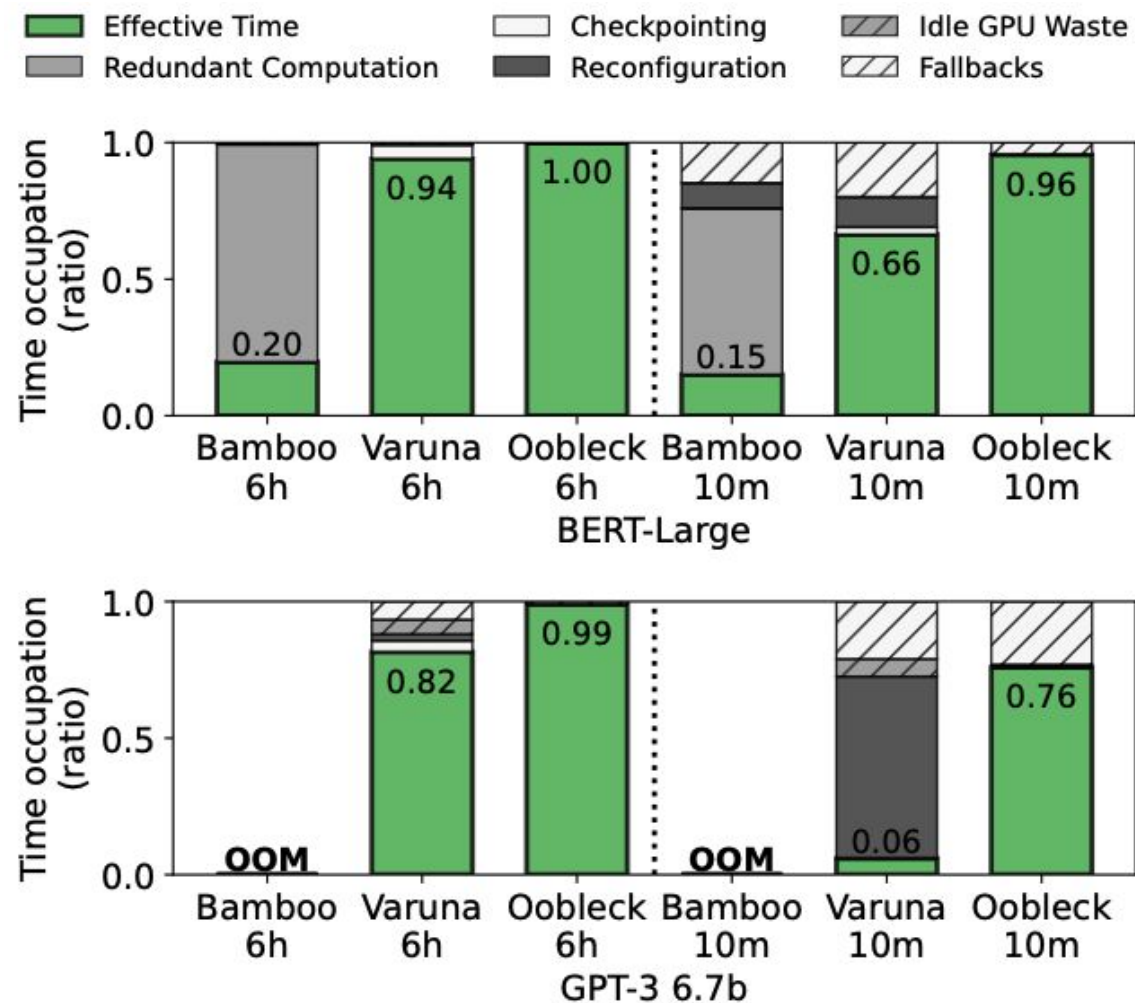
If there are insufficient nodes, merge two pipelines to form a bigger pipeline

Evaluation: Throughput under Controlled Failures

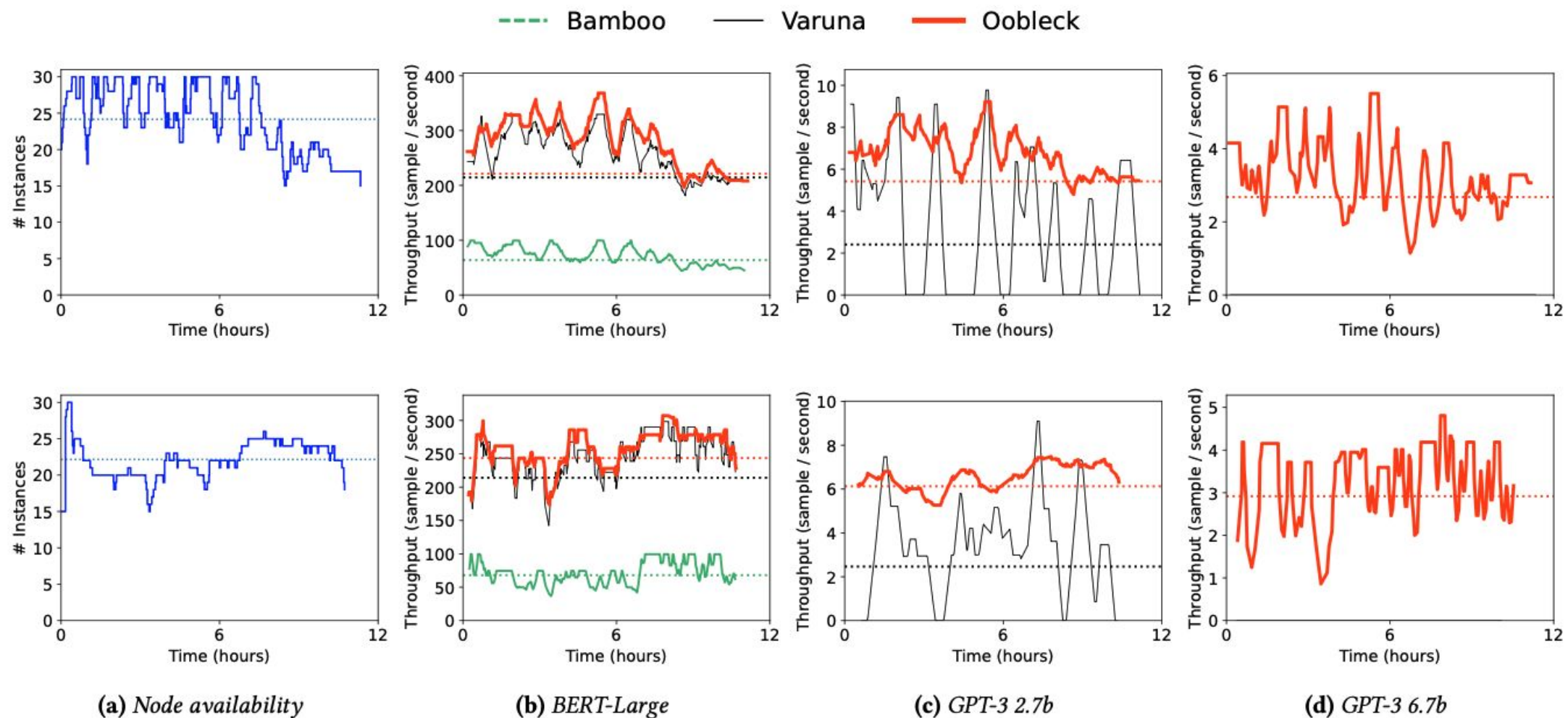
Failure frequency: Once in 6 hours to once in 10 minutes

Model	Throughput	Reason
Bamboo	Consistently low Stable over failure frequencies	Static overhead from redundant computation
Varuna	Same as Oobleck for infrequent failures Low for large models and frequent failures	Fallback overhead from loading and saving checkpoints
Oobleck	Consistently high	Low overhead

Throughput Distribution



Throughput in Spot Instance Traces



Strengths

- Fast failure recovery (instant template switching)
- Strong fault tolerance guarantees (f failures with $f+1$ replicas)
- Low runtime overhead in steady state
- Oobleck outperforms SotA by up to 29.6x during frequent node failures

Limitations

- Not dynamic
- Replication overhead (memory + compute)
- Requires precomputing templates which can be costly
- Pipeline template assumes homogeneous devices

Extensions

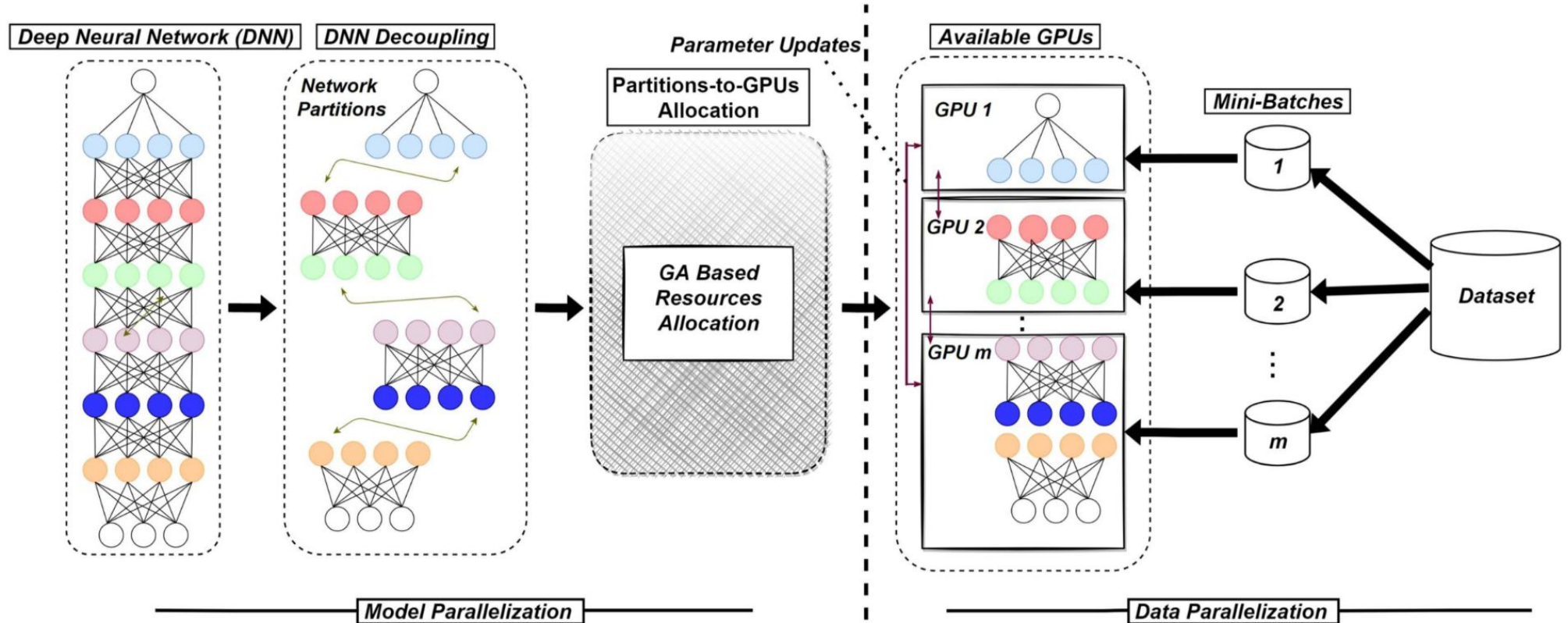
Heterogeneous Hardware Optimization

- Current Oobleck assumes mostly homogeneous GPUs.
- Can be extended to handle **mixed hardware**: GPU + TPU + CPU nodes, adapting batch sizes and parallelism strategies dynamically.



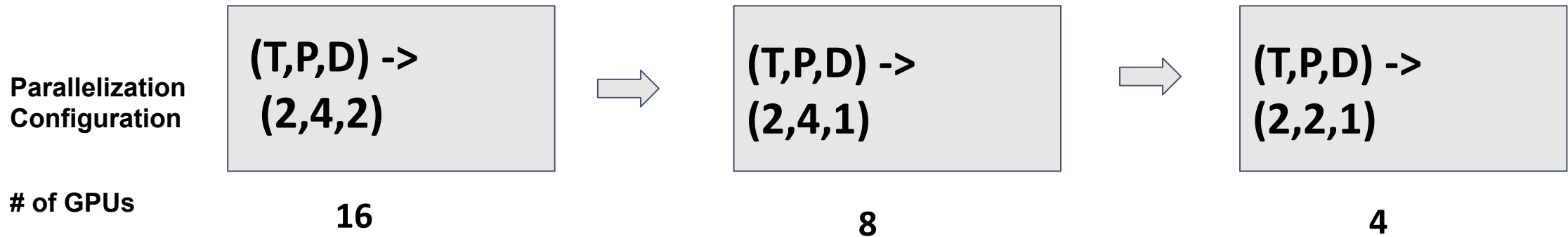
Tenplex: Dynamic Parallelism for Deep Learning using Parallelizable Tensor Collections

Multidimensional Parallelism



What is Dynamic Parallelism?

Dynamic parallelism is **runtime** reconfiguration of DL job parallelization strategies to adapt to changing resources and system conditions.



T -> Tensor , P -> Pipeline, D -> Data

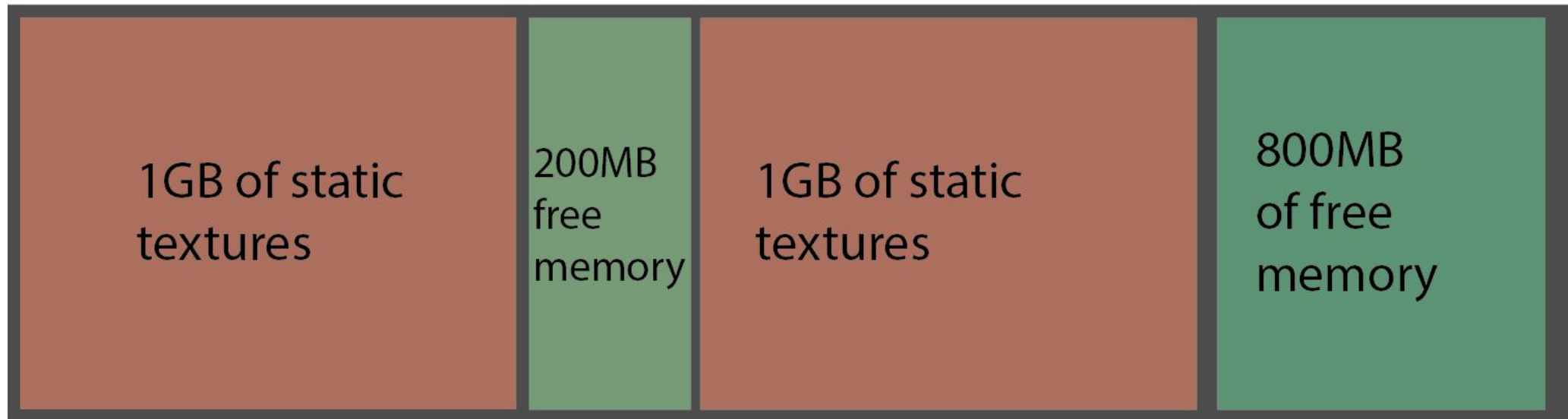
Why Dynamic Parallelism ?

Elasticity - To maintain higher cluster utilization, DL Jobs want to claim extra GPU Resources when the become available.



Why Dynamic Parallelism ?

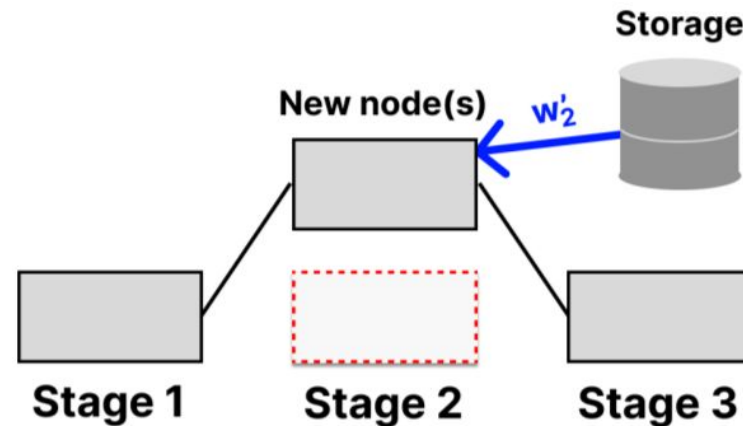
Redeployment - DL jobs may release and migrate GPUs to reduce fragmentation, support maintenance, or handle higher-priority preemption



GPU Fragmentation for 3GB of GPU RAM

Why Dynamic Parallelism ?

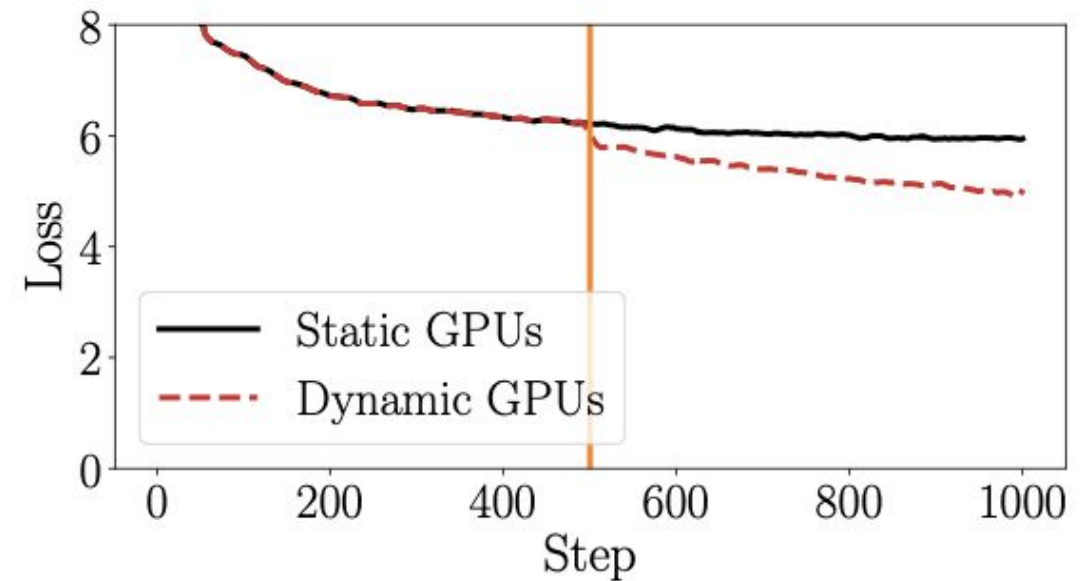
Failure Recovery - Long-running jobs may lose GPUs due to failures, requiring recovery from checkpoints and resuming with new or fewer resources, often forcing suboptimal parallelization.



Challenges in Changing GPU Resources

Impact on Convergence

- Job convergence depends on specific GPUs (not device-independent).
- **Dataset consistency required:** GPU changes affect data sharding
- Inconsistency during re-partitioning can:
 - Duplicate/skip samples mid-epoch.
 - Lead to overfitting

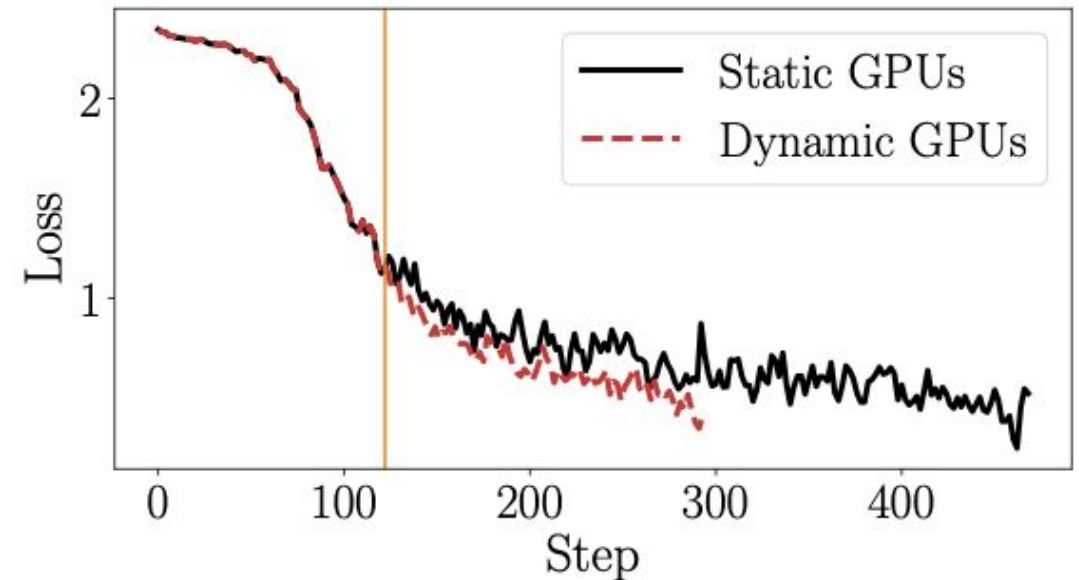


(a) Inconsistent dataset access

Challenges in Changing GPU Resources

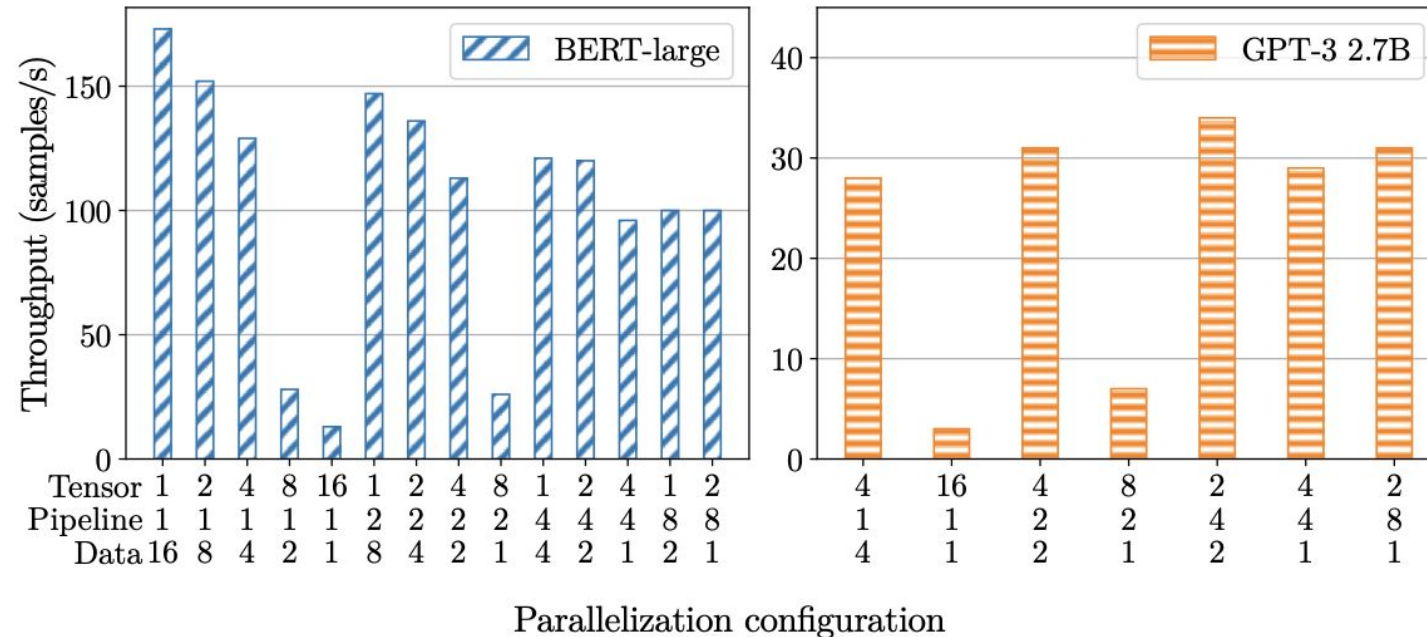
Hyper-Parameter & Batch Size Sensitivity

- Local batch size fixed per GPU; global batch = local $\times$ #GPUs.
- Changing GPUs without adjusting global batch can **destabilize training**.
- Batch size affects gradient updates:
 - Small $\rightarrow$ **noisy, exploratory**
 - Large $\rightarrow$ **accurate, can overshoot**



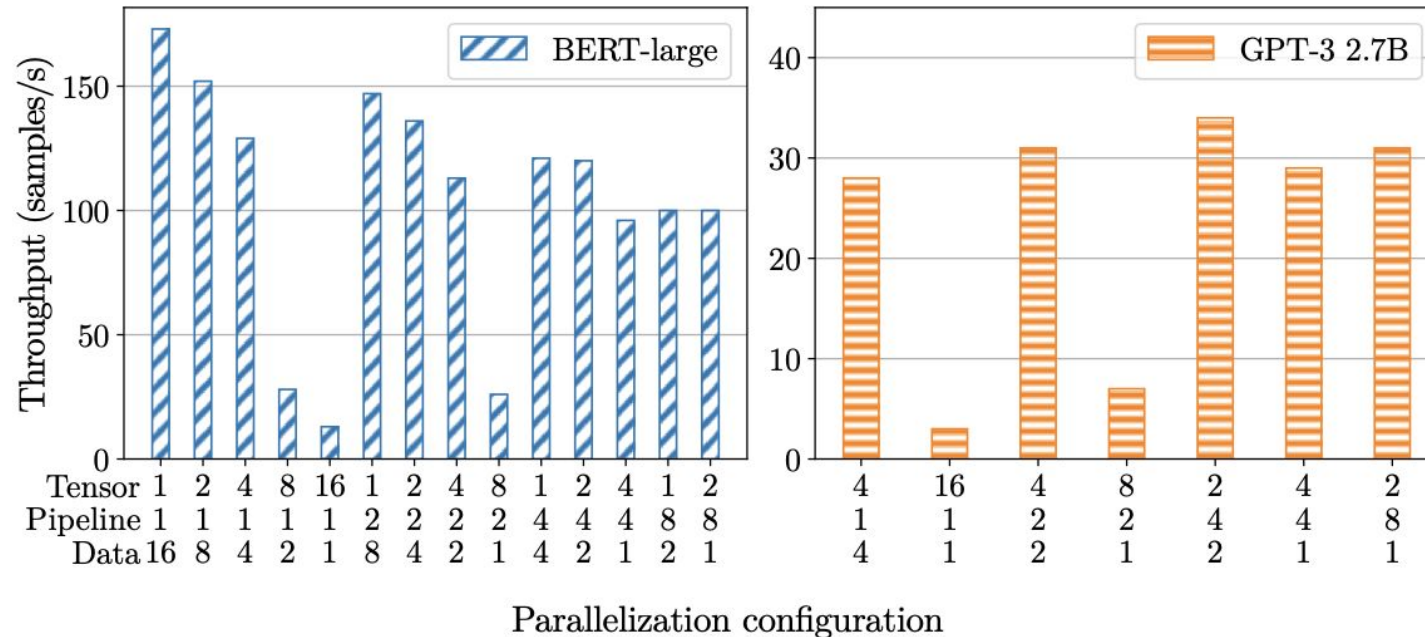
(b) Inconsistent batch size

Challenges in Changing GPU Resources – Performance



Optimal parallelization depends on GPU count & type, Interconnect bandwidth/latency, Model architecture. Changing GPU resources at runtime can make the original configuration **suboptimal**.

Challenges in Changing GPU Resources – Performance



Best config (2,4,2): tensor parallelism limited to fast intra-worker links.

Worst config (16,1,1): tensor parallelism uses slower inter-worker links.

Current Approaches

1. **Model Parallelizers** (Megatron-LM DeepSpeed) - limited to supported DNN architectures, such as specific transformer models
2. **Elastic DL Systems** (Torch Elastic, Elastic Horovod) - Can adapt the number of model replicas at runtime and are limited to data parallel DL Jobs only.
3. **Virtual Devices** (VirtualFlow, EasyScale) - Decouple DL Jobs from physical devices. requires complex virtualization at the GPU driver level and does not support changes with multi-dimensional parallelization

Tenplex

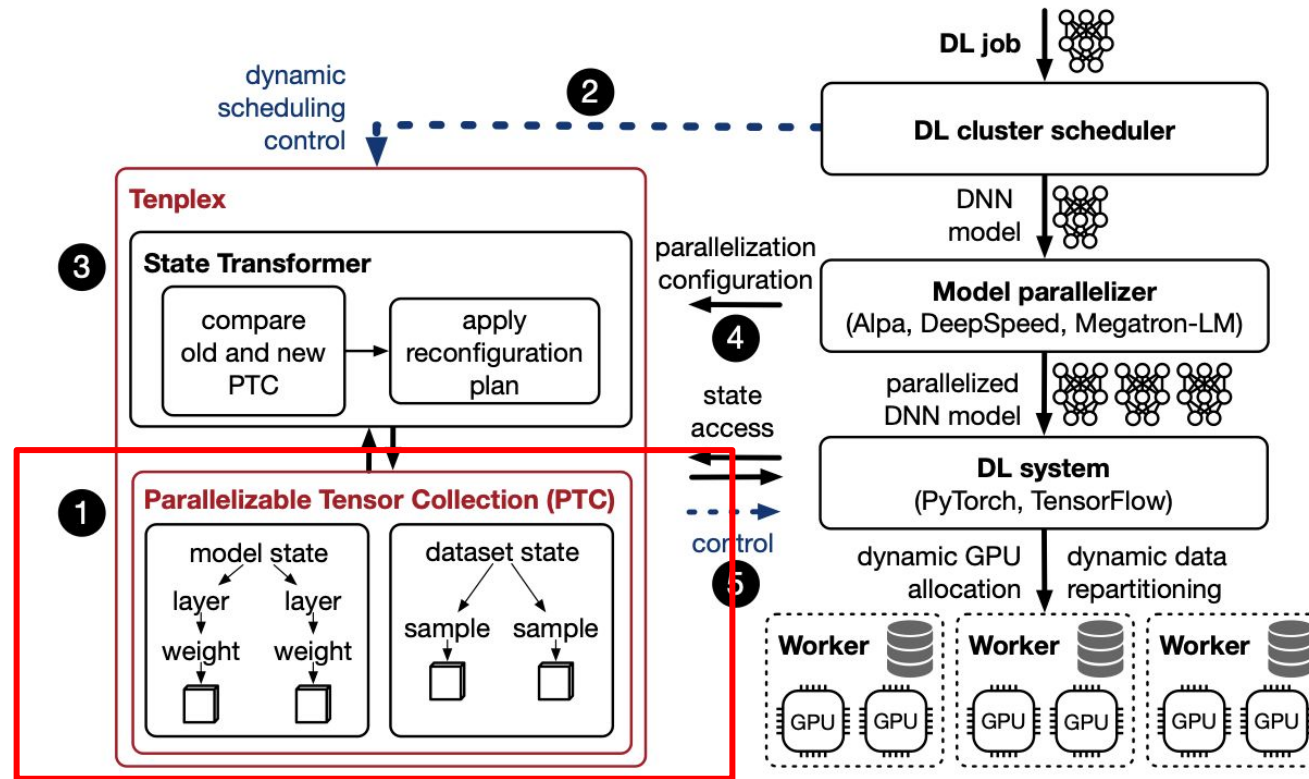
Dynamic State Management for DL Jobs

Goal: Enable multi-dimensional parallel DL jobs to **dynamically change job state** during training in response to GPU changes

- Ensuring consistent training results
- Supporting arbitrary multi-dimensional parallelism
- Minimizing reconfiguration overhead



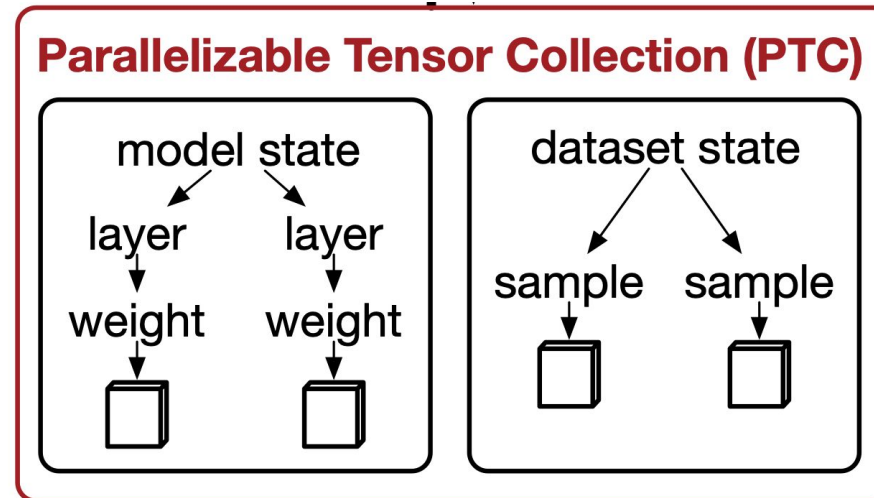
Design & Architecture



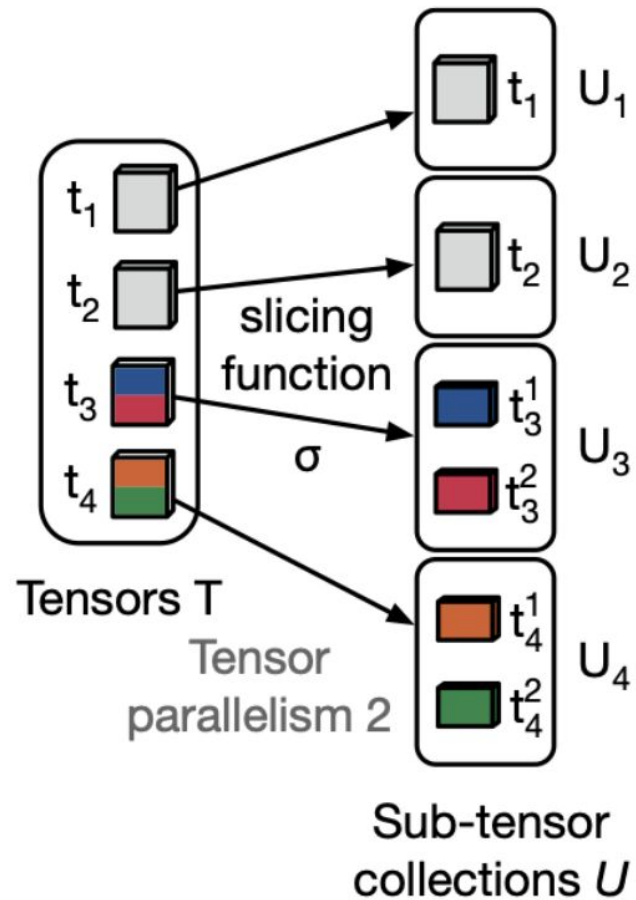
Design & Architecture

1. Externalizing Job State

- Represents dataset & model state as **Parallelizable Tensor Collection (PTC)**
- Hierarchical tensor abstraction
- Virtual file system in-memory



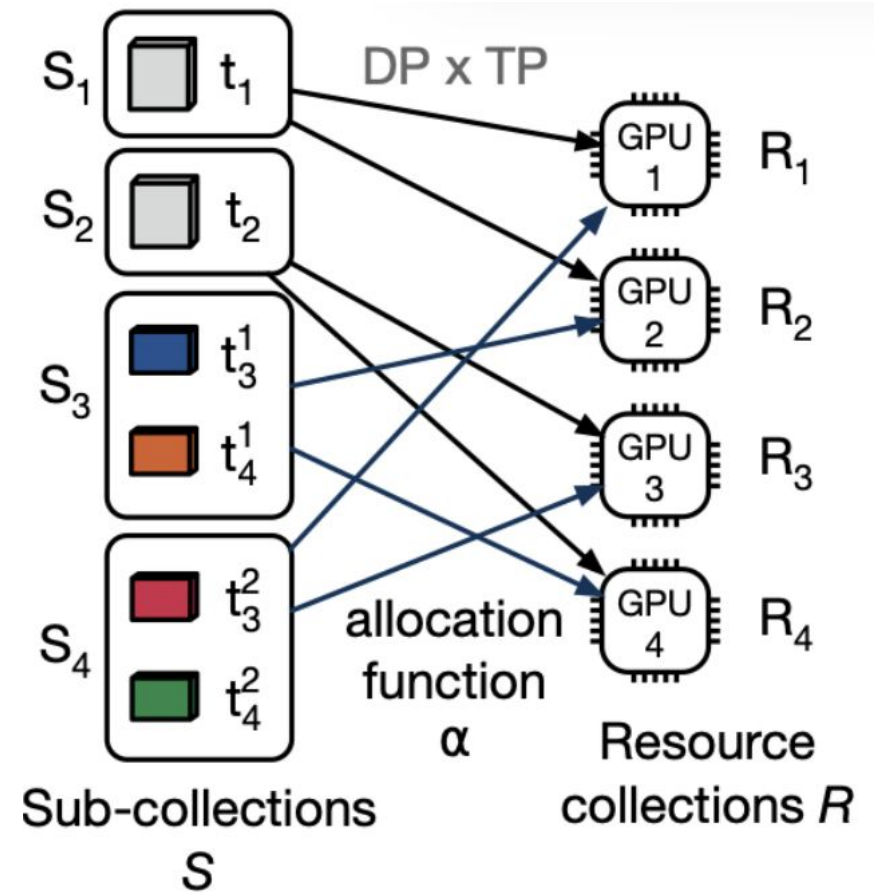
Parallelizable Tensor Collection



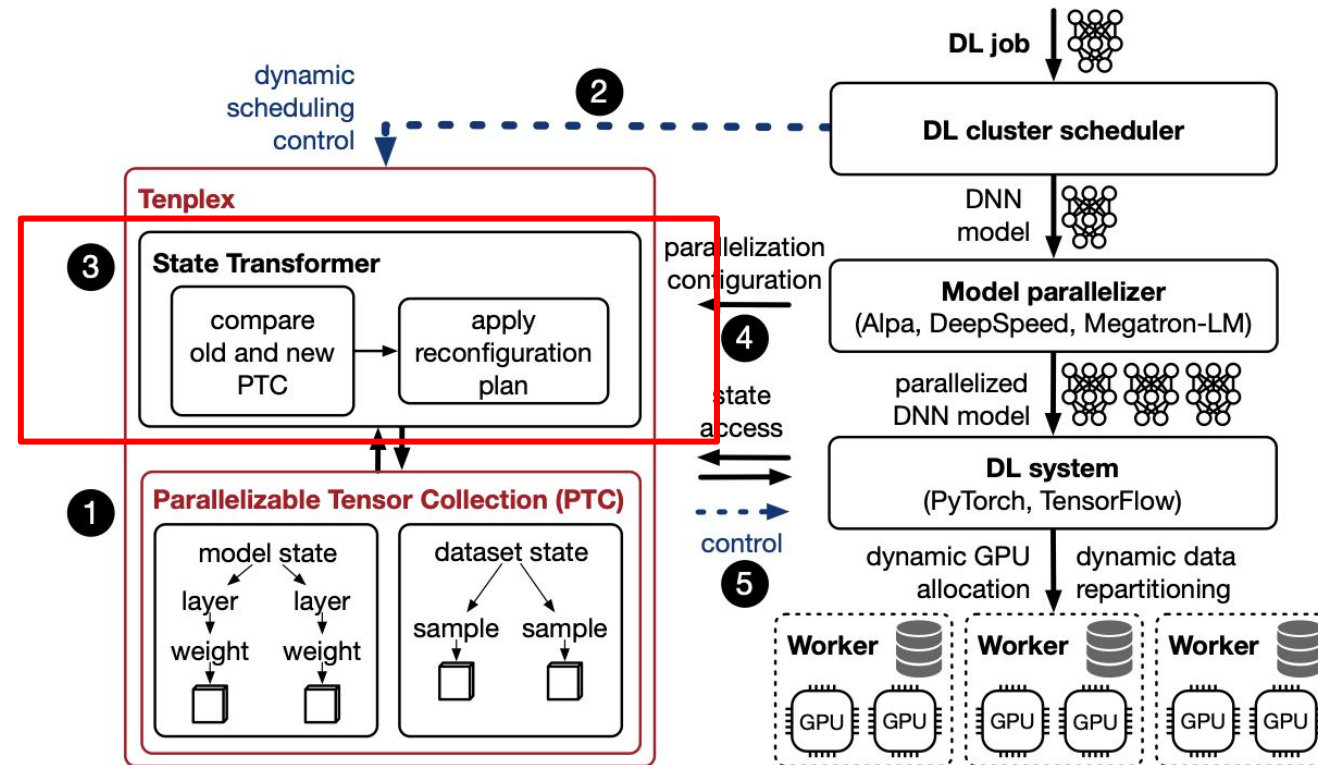
Data
parallelism 2

partitioning
function

ϕ



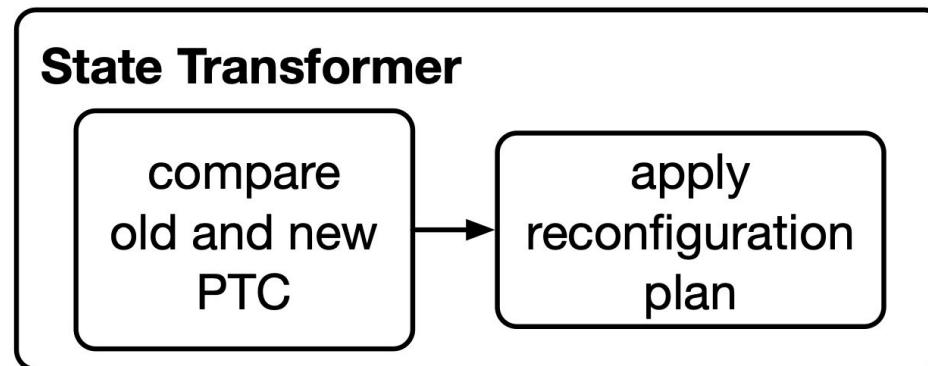
Design & Architecture



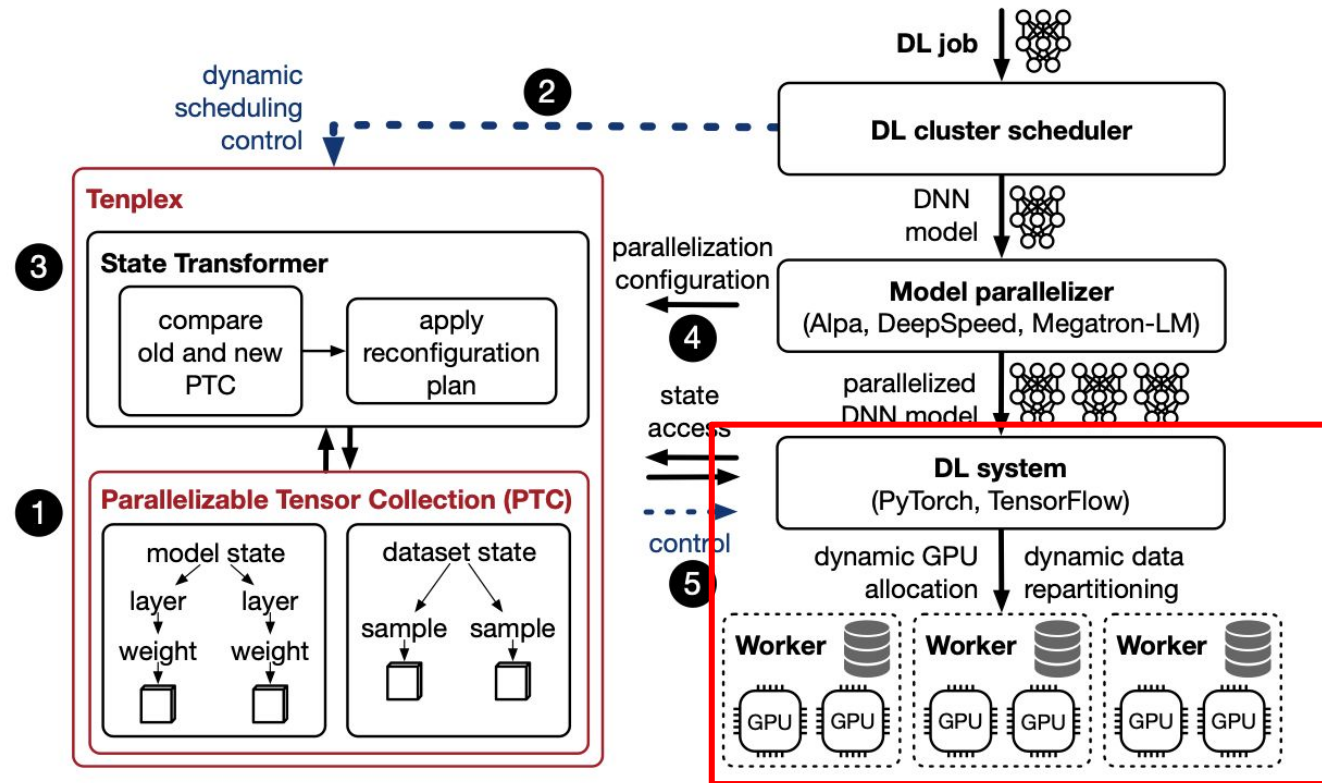
Design & Architecture

2. State Transformation

- Scheduler notifies Tenplex of GPU changes
- **State Transformer** requests new parallelization config (e.g., from Alpa or Megatron-LM)
- Generates a **reconfiguration plan** to repartition and redistribute dataset & model tensors. Maintains training consistency and convergence



Design & Architecture

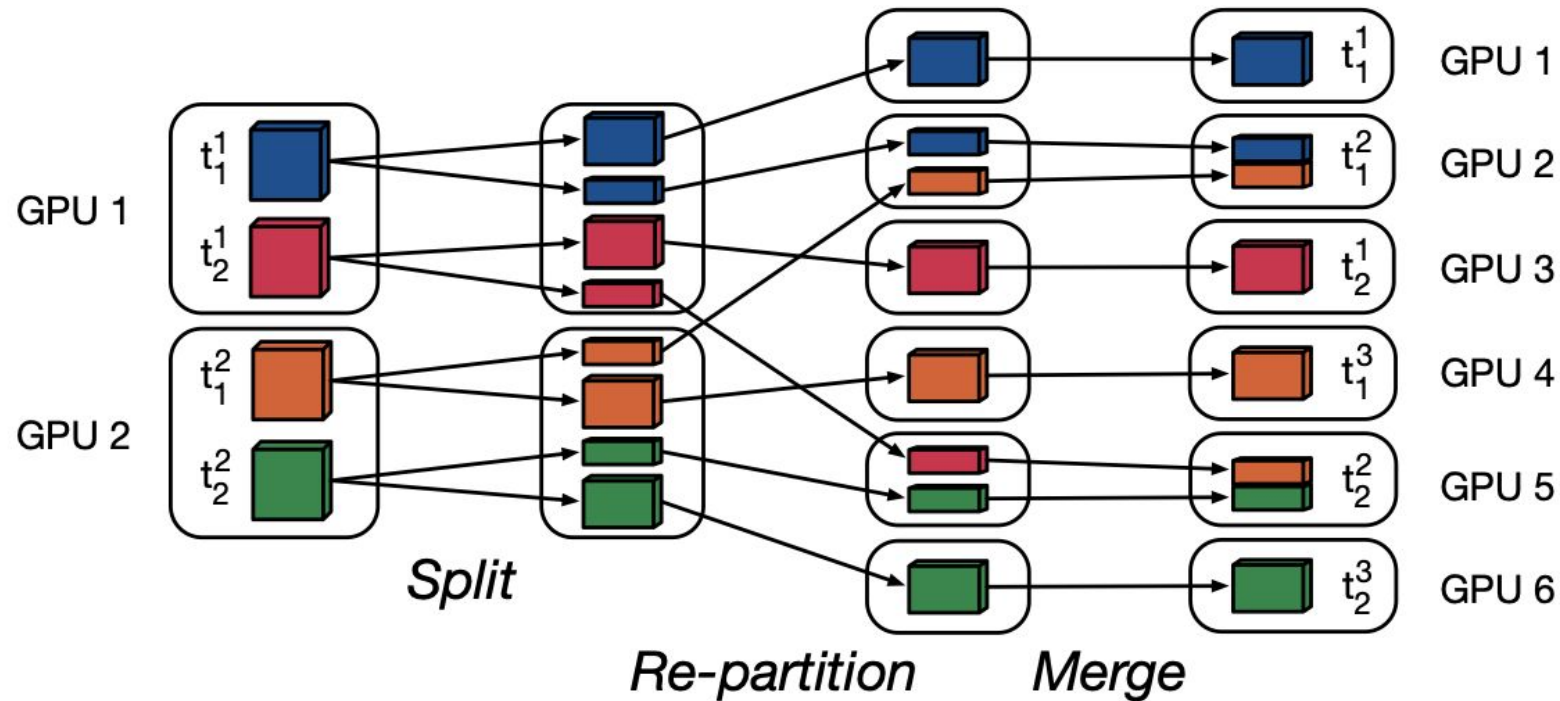


Design & Architecture

3. Optimized Reconfiguration

- Parallelizes transformations across workers, each workers fetch sub-tensors
- Minimizes data movement and overlaps with training
- Supports **any parallelization configuration**, not just data parallelism

Reconfiguration Algorithm



Key Idea:

Reconfiguration plan = **split**
 → **move** → **merge**,
 derived from differences in
 PTC $[(\sigma, \phi, \alpha)]$ vs PTC' $[(\sigma', \phi', \alpha')]$.

Evaluation

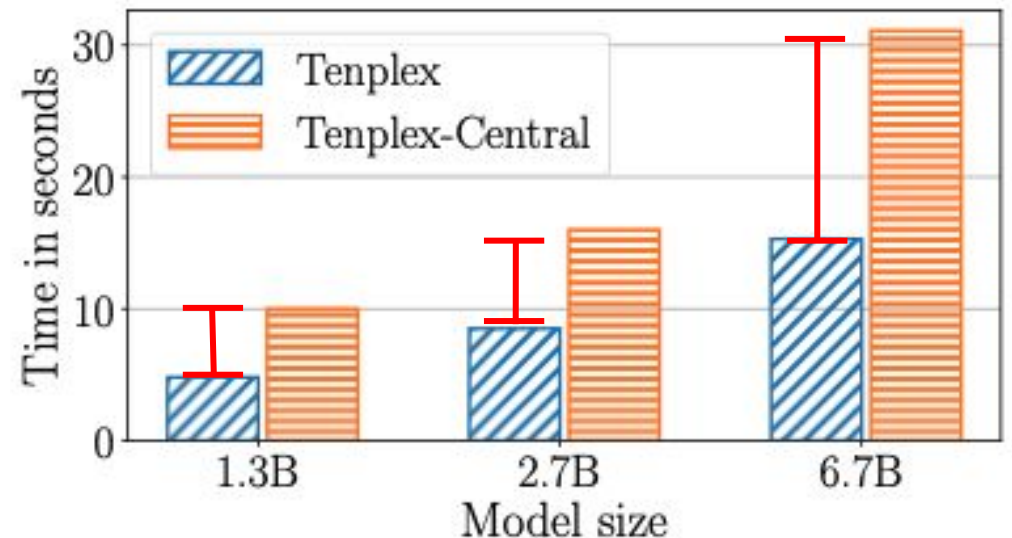
Benchmarks:

1. Torch Distributed Elastic v2.0
2. Horovod Elastic v0.28
3. DeepSpeed v0.6 with Megatron-LM v23.06
4. Tenplex-DP
5. Tenplex-Central

Job Redeployment

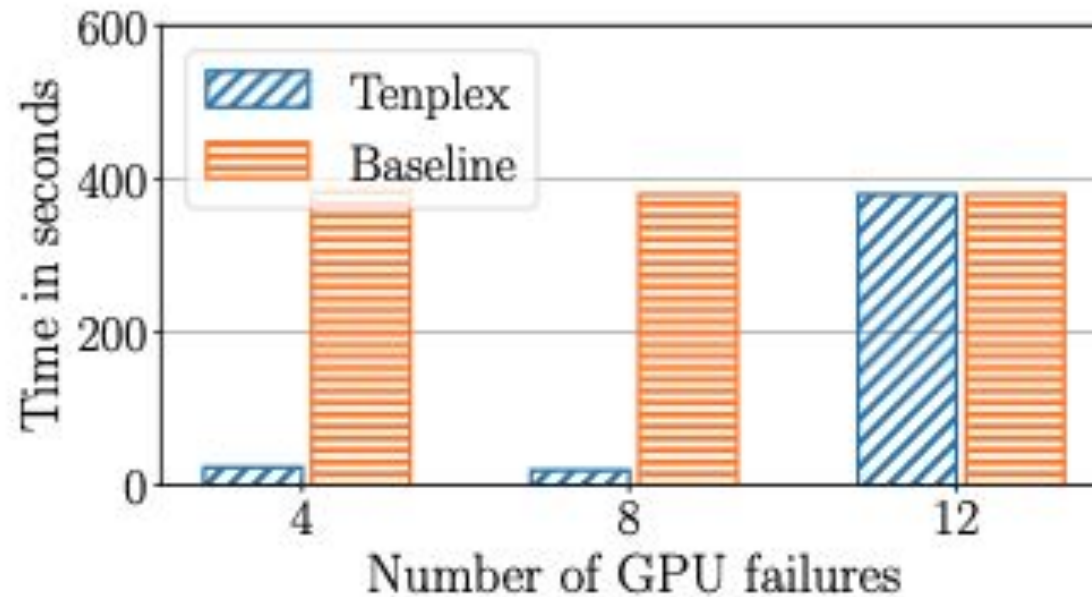
Time taken to redeploy DL jobs with different model sizes onto a new set of GPU resources

- Tenplex Central holds all DL job states at a single central worker
- Tenplex outperforms Tenplex-Central
 - Distributed state management allows migration of state directly between workers
 - Prevents bottleneck caused by network bandwidth of a single worker



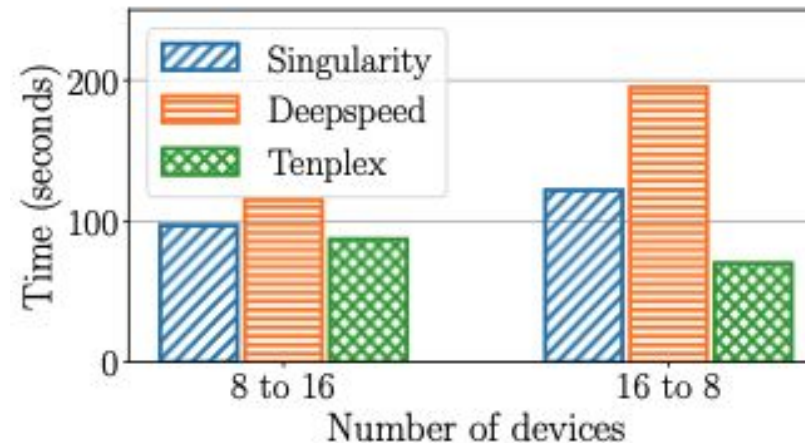
Failure Recovery

- 4-8 GPUs: Tenplex is faster
 - Do not need to re-run lost training steps
- 8 GPUs: 5% recovery time of baseline
 - Same cost as 12 GPUs



Reconfiguration Overhead

Framework	Reconfiguration Time	Reason
Singularity	High	Moves full GPU device state
DeepSpeed	Lower than Singularity	Uses its failure detection mechanism instead of an reconfiguration notification mechanism
Tenplex	Up to 64% less time when scaling from 16-8 GPUs	Aware of data locality due to state management approach



Strengths

- True dynamic parallelism (data + model + pipeline + hybrid)
- Elastic: adapts to GPU scale up/down
- No need for precomputed templates (uses PTCs)
- Maximizes GPU utilization in elastic clusters

Limitations

- Slower recovery than Oobleck (runtime state re-sharding)
- Higher runtime complexity (PTC management)
- Less predictable performance (many configs possible)
- No formal fault tolerance guarantees (not resilient when failures are frequent)

Extensions

Guarantee Fault Tolerance

- Tenplex does not guarantee fault tolerance and could be extended to ensure that.

Tenplex vs. Oobleck

Aspect	Tenplex	Oobleck
Failure handling	Uses replication of model state across workers	Uses pre-planned templates to reassign work
Overhead	Extra memory + communication (since each state is stored multiple times)	Extra pre-computation time (for multiple valid pipeline templates)
Speed of recovery	Recovery is fast (just copy state from replica) but uses up memory	Recovery is fast (just switch template) but requires smart precomputation
Best for	Clusters where failures are less frequent but data integrity is critical and GPU resources change frequently at runtime.	Clusters where failures are frequent and quick reconfiguration is needed